

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

BỘ MÔN LẬP TRÌNH WEB



**BÁO CÁO ĐỒ ÁN PHÁT TRIỂN WEB
E-COMMERCE (TECH WEB)**

Giảng viên hướng dẫn : TRẦN HOÀNG NAM

Nhóm : 10

Họ và tên thành viên : PHẠM MINH SÁNG N23DCCN051

: NGUYỄN VĂN QUYẾN N23DCCN049

: TRẦN BÁ NHẬT N23DCCN043

Lớp : D23CQCN01-N

Hồ Chí Minh – 06/2026

BÁO CÁO ĐỒ ÁN PHÁT TRIỂN ỨNG DỤNG WEB

TechShop

Hệ thống bán hàng công nghệ trực tuyến (Laptop – Điện thoại – Phụ kiện)

Thông tin	Nội dung
Repository	github.com/PiupiuTenshi/TechWeb-2026
Backend	https://techshop-backend-8sfu.onrender.com (Render – Docker Web Service)
Frontend	https://techshop-55gp.vercel.app (Vercel – Vite/React SPA)
Cơ sở dữ liệu	PostgreSQL trên Supabase (qua Entity Framework Core / Npgsql)
Nhóm thực hiện (3 thành viên)	Phạm Minh Sáng (Backend), Nguyễn Văn Quyển (Frontend), Trần Bá Nhật (DevOps)
Thời gian báo cáo	Tháng 6 / 2026

Phân công vai trò nhóm thực hiện

Vai trò	Nhiệm vụ chính
Backend Developer Phạm Minh Sáng	ASP.NET Core Web API, EF Core, JWT, Docker, triển khai Render
Frontend Developer Nguyễn Văn Quyển	React + Vite, tích hợp API, Zustand/React Query, triển khai Vercel
Database / DevOps Trần Bá Nhật	Thiết kế ERD, seed data, biến môi trường, CI/CD GitHub ↔ Render ↔ Vercel

MỤC LỤC

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI.....	6
1.1. Giới thiệu đề tài.....	6
1.2. Mục tiêu đề tài.....	6
1.3. Phạm vi đề tài.....	6
1.4. Đối tượng sử dụng hệ thống.....	7
1.5. Phân công công việc.....	7
CHƯƠNG 2: PHÂN TÍCH YÊU CẦU HỆ THỐNG.....	9
2.1. Yêu cầu chức năng.....	9
2.1.1. Nhóm Xác thực & Tài khoản.....	9
2.1.2. Nhóm Danh mục & Sản phẩm.....	9
2.1.3. Nhóm Giỏ hàng & Đặt hàng.....	9
2.1.4. Nhóm Khuyến mãi & Đánh giá.....	10
2.1.5. Nhóm Quản trị (Admin Panel).....	10
2.2. Yêu cầu phi chức năng.....	10
2.3. Tổng quan các luồng nghiệp vụ chính.....	11
CHƯƠNG 3: CÔNG NGHỆ SỬ DỤNG.....	12
3.1. Kiến trúc tổng thể.....	12
3.2. Công nghệ phía Backend.....	12
3.3. Công nghệ phía Frontend.....	13
3.4. Công nghệ hạ tầng & triển khai.....	13
CHƯƠNG 4: THIẾT KẾ HỆ THỐNG.....	15
4.1. Kiến trúc xử lý request tổng thể.....	15
4.2. Thiết kế cơ sở dữ liệu.....	15
4.2.1. Nhóm Người dùng & Xác thực.....	17
4.2.2. Nhóm Danh mục & Sản phẩm.....	17
4.2.3. Nhóm Giỏ hàng.....	17
4.2.4. Nhóm Đơn hàng & Thanh toán.....	17
4.2.5. Nhóm Khuyến mãi, Kho & Đánh giá.....	18
4.3. Ràng buộc & quy ước thiết kế cơ sở dữ liệu.....	18
4.4. Thiết kế API & chuẩn hóa response.....	18
4.5. Thiết kế bảo mật.....	19
CHƯƠNG 5: TRIỂN KHAI CHỨC NĂNG (IMPLEMENTATION).....	21
5.1. Cấu trúc thư mục Backend.....	21
5.2. Cấu trúc thư mục Frontend.....	21
5.3. Các chức năng chính đã triển khai.....	22
5.4. Danh sách đầy đủ API endpoint.....	22
5.4.1. AuthController — /api/auth.....	22
5.4.2. ProductsController — /api/products.....	23
5.4.3. CategoriesController & BrandsController.....	23
5.4.4. CartController — /api/cart.....	23
5.4.5. OrdersController — /api/orders (yêu cầu đăng nhập).....	24
5.4.6. PaymentsController — /api/payments.....	24
5.4.7. PromotionsController — /api/promotions.....	24
5.4.8. ReviewsController.....	24

5.4.9. AdminOrdersController — /api/admin/orders.....	25
5.4.10. AdminInventoryController — /api/admin/inventory.....	25
5.4.11. AdminReportsController — /api/admin/reports.....	25
5.4.12. AdminUsersController — /api/admin/users.....	25
5.4.13. UsersController — /api/users (tài khoản cá nhân).....	26
5.5. Dữ liệu mẫu (Seed data).....	26
CHƯƠNG 6: TRIỂN KHAI HỆ THỐNG LÊN RENDER VÀ VERCEL.....	27
6.1. Tổng quan chiến lược triển khai.....	27
6.2. Triển khai Backend lên Render.....	27
6.2.1. Các tệp cấu hình triển khai có sẵn trong repository.....	27
6.2.2. Các bước triển khai chi tiết trên Render.....	29
6.2.3. Bảng biến môi trường Backend trên Render.....	29
6.2.4. Cấu hình cơ sở dữ liệu PostgreSQL trên Supabase (môi trường Production).....	31
6.2.5. Quy trình build & deploy tự động (CI/CD trên Render).....	31
6.2.6. Giới hạn của gói Free trên Render và cách xử lý.....	32
6.2.7. Kiểm tra sau khi triển khai.....	32
6.3. Triển khai Frontend lên Vercel.....	32
6.3.1. Chuẩn bị trước khi triển khai.....	32
6.3.2. Các bước triển khai chi tiết trên Vercel.....	33
6.3.3. Bảng biến môi trường Frontend trên Vercel.....	33
6.3.4. Cấu hình SPA Fallback (vercel.json).....	34
6.3.5. Auto-Deploy và Preview Deployment.....	34
6.3.6. Custom Domain (tùy chọn mở rộng).....	34
6.4. Kết nối Frontend và Backend sau khi triển khai (CORS).....	34
6.5. Sơ đồ luồng triển khai tổng thể.....	35
6.6. Bảng tổng hợp cấu hình triển khai.....	35
6.7. Một số vấn đề thường gặp khi triển khai và cách xử lý.....	36
6.8. Chạy toàn bộ hệ thống bằng Docker Compose (môi trường phát triển local).....	37
CHƯƠNG 7: KIỂM THỬ HỆ THỐNG.....	38
7.1. Công cụ và tài nguyên kiểm thử có sẵn.....	38
7.2. Kịch bản kiểm thử luồng nghiệp vụ đầy đủ.....	38
7.3. Tài khoản kiểm thử mặc định.....	38
7.4. Một số trường hợp kiểm thử tiêu biểu (Test Case).....	39
CHƯƠNG 8: KẾT QUẢ ĐẠT ĐƯỢC VÀ HẠN CHẾ.....	40
8.1. Kết quả đạt được.....	40
8.2. Hạn chế hiện tại.....	40
CHƯƠNG 9: HƯỚNG PHÁT TRIỂN.....	41
CHƯƠNG 10: KẾT LUẬN.....	42
TÀI LIỆU THAM KHẢO.....	43

DANH MỤC HÌNH

Hình 1 — Sơ đồ thực thể liên kết (ERD) cơ sở dữ liệu hệ thống TechShop.....	16
---	----

DANH MỤC BẢNG

Bảng 1: Đối tượng sử dụng hệ thống.....	7
Bảng 2: Phân công công việc.....	7
Bảng 3: Nhóm Xác thực & Tài khoản.....	9
Bảng 4: Nhóm Danh mục & Sản phẩm.....	9
Bảng 5: Nhóm Giỏ hàng & Đặt hàng.....	9
Bảng 6: Nhóm Khuyến mãi & Đánh giá.....	10
Bảng 7: Nhóm Quản trị (Admin Panel).....	10
Bảng 8: Yêu cầu phi chức năng.....	10
Bảng 9: Công nghệ phía Backend.....	12
Bảng 10: Công nghệ phía Frontend.....	13
Bảng 11: Công nghệ hạ tầng & triển khai.....	13
Bảng 12: Nhóm Người dùng & Xác thực.....	17
Bảng 13: Nhóm Danh mục & Sản phẩm.....	17
Bảng 14: Nhóm Giỏ hàng.....	17
Bảng 15: Nhóm Đơn hàng & Thanh toán.....	17
Bảng 16: Nhóm Khuyến mãi, Kho & Đánh giá.....	18
Bảng 17: Thiết kế API & chuẩn hóa response.....	19
Bảng 18: Các chức năng chính đã triển khai.....	22
Bảng 19: AuthController — /api/auth.....	23
Bảng 20: ProductsController — /api/products.....	23
Bảng 21: CategoriesController & BrandsController.....	23
Bảng 22: CartController — /api/cart.....	23
Bảng 23: OrdersController — /api/orders (yêu cầu đăng nhập).....	24
Bảng 24: PaymentsController — /api/payments.....	24
Bảng 25: PromotionsController — /api/promotions.....	24
Bảng 26: ReviewsController.....	24
Bảng 27: AdminOrdersController — /api/admin/orders.....	25
Bảng 28: AdminInventoryController — /api/admin/inventory.....	25
Bảng 29: AdminReportsController — /api/admin/reports.....	25
Bảng 30: AdminUsersController — /api/admin/users.....	25
Bảng 31: UsersController — /api/users (tài khoản cá nhân).....	26
Bảng 32: Dữ liệu mẫu (Seed data).....	26
Bảng 33: Biến môi trường Backend trên Render.....	30
Bảng 34: Biến môi trường Frontend trên Vercel.....	33
Bảng 35: Kết nối Frontend và Backend sau khi triển khai (CORS).....	34
Bảng 36: tổng hợp cấu hình triển khai.....	35
Bảng 37: Một số vấn đề thường gặp khi triển khai và cách xử lý.....	36
Bảng 38: Các dịch vụ định nghĩa trong docker-compose.yml.....	37
Bảng 39: Tài khoản kiểm thử mặc định.....	38
Bảng 40: Một số trường hợp kiểm thử tiêu biểu (Test Case).....	39
Bảng 41: Hạn chế hiện tại.....	40

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

1.1. Giới thiệu đề tài

TechShop là một ứng dụng web thương mại điện tử chuyên kinh doanh các sản phẩm công nghệ như laptop, điện thoại di động và phụ kiện. Hệ thống được xây dựng theo mô hình tách biệt giữa phần giao diện người dùng (Frontend) và phần xử lý nghiệp vụ (Backend API), giao tiếp với nhau thông qua chuẩn RESTful API. Đây là kiến trúc phổ biến trong các hệ thống web hiện đại, cho phép hai phần phát triển, kiểm thử và triển khai độc lập với nhau.

Báo cáo này trình bày toàn diện quá trình phân tích, thiết kế, xây dựng và đặc biệt là triển khai hệ thống lên môi trường thực tế, sử dụng hai dịch vụ cloud miễn phí phổ biến hiện nay là Render (cho Backend API, dưới dạng container Docker) và Vercel (cho Frontend), kết hợp với cơ sở dữ liệu PostgreSQL được quản lý trên cloud.

Toàn bộ mã nguồn của đề tài được quản lý trên GitHub tại địa chỉ repository [PiupiuTenshi/TechWeb-2026](#)

1.2. Mục tiêu đề tài

Đề tài hướng tới các mục tiêu cụ thể sau:

- Xây dựng một hệ thống bán hàng trực tuyến hoàn chỉnh, mô phỏng đầy đủ vòng đời mua sắm: duyệt sản phẩm, tìm kiếm/lọc/sắp xếp, thêm vào giỏ hàng, áp dụng mã giảm giá, thanh toán và theo dõi đơn hàng.
- Áp dụng cơ chế xác thực và phân quyền theo vai trò (RBAC) với ba vai trò: Khách vãng lai, Khách hàng, Nhân viên và Quản trị viên.
- Xây dựng phân hệ quản trị cho phép quản lý sản phẩm, kho hàng, đơn hàng, khuyến mãi, người dùng và xem báo cáo doanh thu.
- Thiết kế cơ sở dữ liệu quan hệ chuẩn hóa, đảm bảo tính nhất quán dữ liệu thông qua transaction và các ràng buộc khóa ngoại.
- Đóng gói Backend bằng Docker và triển khai theo mô hình container hóa lên nền tảng Render.
- Triển khai Frontend dưới dạng ứng dụng Single Page Application lên nền tảng Vercel.
- Thiết lập quy trình CI/CD đơn giản: tự động build và deploy lại hệ thống mỗi khi có thay đổi mã nguồn được đẩy lên nhánh chính (main) trên GitHub.

1.3. Phạm vi đề tài

Phạm vi đã hoàn thiện:

- Toàn bộ luồng nghiệp vụ lõi: xác thực người dùng (đăng ký/đăng nhập/đăng nhập Google/refresh token), danh mục sản phẩm, giỏ hàng (cho cả khách và người dùng đã đăng nhập), đặt hàng, hủy đơn, khuyến mãi, đánh giá sản phẩm và toàn bộ phân hệ quản trị.
- Hạ tầng triển khai thực tế trên Render (Backend) và quy trình triển khai chi tiết lên Vercel (Frontend).

Phạm vi giới hạn / mô phỏng:

- Bốn cổng thanh toán VNPAY, MoMo, VietQR và ZaloPay hiện đang ở dạng mock: hệ thống tự sinh URL/callback có ký HMAC-SHA256 thay vì gọi API thật của các cổng thanh toán. Tương lai sẽ tích hợp api của cổng thanh toán thật.
- Dịch vụ gửi email xác nhận đơn hàng/thanh toán hiện vẫn ghi log ra console thay vì gửi email thật; riêng chức năng "Quên mật khẩu" (forgot-password) đã gửi email thật chứa mật khẩu mới qua Brevo Transactional Email API.
- Chưa tích hợp đơn vị vận chuyển thật; trường mã vận đơn (TrackingCode) được nhập thủ công bởi nhân viên quản trị.

1.4. Đối tượng sử dụng hệ thống

Bảng 1: Đối tượng sử dụng hệ thống

Vai trò	Mô tả quyền hạn
Khách vãng lai	Chưa đăng nhập. Có thể xem danh sách/chi tiết sản phẩm, tìm kiếm, lọc theo danh mục/hãng/giá, thêm sản phẩm vào giỏ hàng tạm (định danh qua header X-Session-Id).
Khách hàng	Đã đăng ký/đăng nhập (email-password hoặc Google OAuth). Có đầy đủ quyền của Guest, cộng thêm: quản lý địa chỉ giao hàng, đặt hàng (COD/VNPAY/MoMo/VietQR/ZaloPay mock), xem lịch sử đơn hàng, hủy đơn (khi chưa hoàn tất), viết đánh giá sản phẩm sau khi đơn hàng ở trạng thái Completed.
Nhân viên	Đăng nhập với vai trò Staff. Có quyền truy cập phân hệ quản trị: quản lý sản phẩm, quản lý tồn kho (nhập/xuất/điều chỉnh), cập nhật trạng thái và mã vận đơn, xem báo cáo doanh thu/sản phẩm bán chạy/tồn kho thấp.
Quản trị viên	Có toàn bộ quyền của Staff, cộng thêm quyền quản lý người dùng (tạo, cập nhật, khóa/mở tài khoản, đổi mật khẩu, xóa mềm) và quản lý chương trình khuyến mãi.

1.5. Phân công công việc

Theo định hướng phân chia dự án được tổ chức thành ba mảng công việc chính:

Bảng 2: Phân công công việc

Vai trò	Phụ trách	Nhiệm vụ chính
Backend Developer	Phát triển .NET API Phạm Minh Sáng	Xây dựng Controllers, Models, DTOs, Services; cấu hình xác thực JWT, áp dụng Entity Framework Core, viết migration, đóng gói Docker và cấu hình

		triển khai Render.
Frontend Developer	Phát triển giao diện React Nguyễn Văn Quyến	Xây dựng các trang (pages), component dùng lại, tích hợp gọi API thật bằng Axios/Fetch, quản lý state với Zustand và React Query, xây dựng phân hệ quản trị, triển khai lên Vercel.
Database / DevOps	Thiết kế CSDL & vận hành Trần Bá Nhật	Thiết kế lược đồ cơ sở dữ liệu (ERD), chuẩn hóa bảng, viết seed data, cấu hình PostgreSQL, theo dõi biến môi trường, dựng quy trình CI/CD giữa GitHub – Render – Vercel.

Dự án được thực hiện bởi nhóm ba thành viên: Phạm Minh Sáng (Backend Developer), Nguyễn Văn Quyến (Frontend Developer) và Trần Bá Nhật (Database / DevOps); vai trò công việc được tổ chức và phân chia theo mô hình nhóm ba thành viên như mô tả ở trên.

CHƯƠNG 2: PHÂN TÍCH YÊU CẦU HỆ THỐNG

2.1. Yêu cầu chức năng

Yêu cầu chức năng được tổng hợp theo từng nhóm nghiệp vụ, tương ứng với các controller trong Backend và các trang/route trong Frontend.

2.1.1. Nhóm Xác thực & Tài khoản

Bảng 3: Nhóm Xác thực & Tài khoản

Mã	Chức năng	Diễn giải
FR-01	Đăng ký tài khoản	Khách tạo tài khoản Customer mới bằng email/mật khẩu; hệ thống kiểm tra email trùng, mã hóa mật khẩu bằng BCrypt.
FR-02	Đăng nhập	Đăng nhập bằng email/mật khẩu hoặc Google OAuth (Google.Apis.Auth), trả về accessToken (JWT) và refreshToken.
FR-03	Làm mới token	Khi accessToken hết hạn, frontend gọi API refresh để cấp accessToken mới mà không cần đăng nhập lại.
FR-04	Đăng xuất	Vô hiệu hóa, refreshToken hiện tại.
FR-05	Quản lý địa chỉ	Người dùng thêm/sửa địa chỉ nhận hàng, đặt địa chỉ mặc định.

2.1.2. Nhóm Danh mục & Sản phẩm

Bảng 4: Nhóm Danh mục & Sản phẩm

Mã	Chức năng	Diễn giải
FR-06	Xem danh mục	Lấy cây danh mục cha/con (Category) để dựng menu điều hướng và bộ lọc.
FR-07	Liệt kê sản phẩm	Lọc theo danh mục, hãng (brand), khoảng giá; sắp xếp theo giá tăng/giảm/mới nhất; phân trang.
FR-08	Xem chi tiết sản phẩm	Hiển thị thông tin sản phẩm, ảnh, các biến thể (màu/RAM/dung lượng), thông số kỹ thuật và điểm đánh giá trung bình.
FR-09	Quản lý sản phẩm (Admin/Staff)	Tạo, cập nhật, xóa mềm sản phẩm; tải ảnh sản phẩm.

2.1.3. Nhóm Giỏ hàng & Đặt hàng

Bảng 5: Nhóm Giỏ hàng & Đặt hàng

Mã	Chức năng	Diễn giải
FR-10	Giỏ hàng khách vãng lai	Khách chưa đăng nhập vẫn có giỏ hàng riêng, định danh bằng SessionId lưu trong header X-Session-Id.
FR-11	Thêm/sửa/xóa sản phẩm trong giỏ	Thao tác trên từng ProductVariant cụ thể với số lượng.
FR-12	Áp dụng mã giảm giá	Kiểm tra điều kiện Coupon (giá trị tối thiểu đơn hàng, giới hạn sử dụng, thời hạn hiệu lực).
FR-13	Tạo đơn hàng	Chuyển toàn bộ giỏ hàng thành Order trong một transaction: kiểm tra tồn kho, trừ kho, ghi log kho, tạo Payment ở trạng thái Pending.
FR-14	Hủy đơn hàng	Khách hàng được hủy đơn khi đơn chưa ở trạng thái Completed/Cancelled; hệ thống hoàn lại tồn kho.

FR-15	Thanh toán COD/VNPay/MoMo/VietQR/ZaloPay (mock)	Sinh URL thanh toán mô phỏng, xác thực callback bằng chữ ký HMAC-SHA256.
-------	--	--

2.1.4. Nhóm Khuyến mãi & Đánh giá

Bảng 6: Nhóm Khuyến mãi & Đánh giá

Mã	Chức năng	Diễn giải
FR-16	Quản lý khuyến mãi	Tạo chương trình khuyến mãi áp dụng cho danh sách sản phẩm cụ thể, theo % hoặc số tiền cố định, có thời gian bắt đầu/kết thúc.
FR-17	Viết đánh giá sản phẩm	Khách hàng chỉ được đánh giá sản phẩm đã mua và đơn hàng đã ở trạng thái Completed; đánh giá mới mặc định ẩn (isVisible = false) chờ duyệt.

2.1.5. Nhóm Quản trị (Admin Panel)

Bảng 7: Nhóm Quản trị (Admin Panel)

Mã	Chức năng	Diễn giải
FR-18	Quản lý đơn hàng	Xem danh sách đơn theo trạng thái, cập nhật trạng thái đơn và mã vận đơn.
FR-19	Quản lý tồn kho	Xem tồn kho theo biến thể, nhập/xuất/điều chỉnh kho, xem lịch sử log kho.
FR-20	Báo cáo thống kê	Báo cáo doanh thu theo ngày/tháng, top sản phẩm bán chạy, sản phẩm tồn kho thấp; xuất báo cáo doanh thu ra Excel.
FR-21	Quản lý người dùng	Tạo, cập nhật, khóa/mở, đổi mật khẩu, xóa mềm tài khoản người dùng (chỉ Admin).

2.2. Yêu cầu phi chức năng

Bảng 8: Yêu cầu phi chức năng

Tiêu chí	Mô tả & cách đáp ứng
An toàn / Bảo mật	Mật khẩu được mã hóa một chiều bằng BCrypt; xác thực bằng JWT Bearer với accessToken ngắn hạn (15 phút) và refreshToken dài hạn (7 ngày, có thể revoke); callback thanh toán được xác minh chữ ký HMAC-SHA256 theo cơ chế so sánh thời gian cố định (fixed-time compare) để chống timing attack; CORS chỉ cho phép danh sách origin được khai báo qua biến môi trường.
Tính nhất quán dữ liệu	Luồng tạo đơn hàng được bọc trong một database transaction; nếu bất kỳ bước nào thất bại (hết hàng, sản phẩm không hợp lệ...) toàn bộ thay đổi sẽ được rollback.
Hiệu năng	Danh sách sản phẩm/đơn hàng/người dùng đều hỗ trợ phân trang (page, pageSize); các trường tìm kiếm quan trọng (Email, Slug, SKU, Code) đều có unique index để tăng tốc truy vấn và đảm bảo ràng buộc duy nhất.
Khả năng mở rộng & vận hành	Backend đóng gói dưới dạng container Docker, không phụ thuộc môi trường máy chủ cụ thể; cấu hình (chuỗi kết nối, khóa bí mật JWT, danh sách CORS...) được tách hoàn toàn ra biến môi trường theo nguyên tắc Twelve-Factor App, cho phép triển khai trên nhiều nền tảng cloud khác nhau mà không cần sửa code.
Khả dụng	Backend cung cấp endpoint /health phục vụ cơ chế Health Check tự động của Render, giúp nền tảng phát hiện và khởi động lại container khi gặp sự cố.

Khả năng bảo trì	Tách rõ ba lớp Controller – Service/DTO – Data (EF Core); áp dụng chuẩn response thống nhất ApiResponse<T> cho toàn bộ API giúp Frontend xử lý lỗi/dữ liệu nhất quán.
------------------	---

2.3. Tổng quan các luồng nghiệp vụ chính

Các luồng nghiệp vụ trọng yếu mà hệ thống phải xử lý chính xác bao gồm:

1. Luồng xác thực: Đăng ký → Đăng nhập → Sử dụng accessToken cho các request cần quyền → Tự động refresh khi accessToken hết hạn → Đăng xuất (revoke refreshToken).
2. Luồng duyệt sản phẩm: Lấy cây danh mục và danh sách hãng → Lọc/sắp xếp/phân trang sản phẩm → Xem chi tiết sản phẩm và chọn biến thể.
3. Luồng giỏ hàng khách: Tạo giỏ hàng gắn với SessionId → Thêm sản phẩm → Lưu lại SessionId từ header phản hồi cho các lần gọi sau.
4. Luồng thanh toán COD: Tạo đơn hàng → Trừ tiền kho → Ghi log kho → Tạo Payment trạng thái Pending → Xóa giỏ hàng.
5. Luồng thanh toán VNPAY/MoMo/VietQR/ZaloPay (mock): Tạo đơn hàng → Gọi API tạo URL thanh toán → Chuyển hướng người dùng → Callback xác minh chữ ký → Cập nhật trạng thái Payment và Order.
6. Luồng hủy đơn: Kiểm tra quyền sở hữu và trạng thái hợp lệ → Đổi trạng thái Cancelled → Hoàn tiền kho → Ghi OrderStatusLog.
7. Luồng vận hành kho (Admin/Staff): Nhập/xuất/điều chỉnh tiền kho → Ghi InventoryLog tương ứng với từng loại thay đổi (Import, Export, Adjust, SaleDeduct, CancelReturn).

CHƯƠNG 3: CÔNG NGHỆ SỬ DỤNG

3.1. Kiến trúc tổng thể

Hệ thống được xây dựng theo kiến trúc ba lớp (3-tier architecture) với sự tách biệt rõ ràng giữa tầng giao diện, tầng xử lý nghiệp vụ/API và tầng dữ liệu:

- Tầng giao diện: ứng dụng React chạy hoàn toàn trên trình duyệt, được build thành các tệp tĩnh (HTML/CSS/JS) và phân phối qua Vercel Edge Network.
- Tầng xử lý nghiệp vụ: ASP.NET Core Web API đóng gói trong container Docker, triển khai trên Render dưới dạng Web Service, cung cấp các endpoint RESTful trả về JSON theo chuẩn ApiResponse<T>.
- Tầng dữ liệu: cơ sở dữ liệu quan hệ PostgreSQL, được Backend truy cập thông qua Entity Framework Core với provider Npgsql.

Hai tầng Frontend và Backend giao tiếp với nhau hoàn toàn qua HTTPS bằng các request REST/JSON; Backend ở trạng thái phi trạng thái, xác thực người dùng dựa trên JWT Bearer Token gửi kèm mỗi request, còn giỏ hàng khách được định danh bằng header tùy biến X-Session-Id.

3.2. Công nghệ phía Backend

Bảng 9: Công nghệ phía Backend

Thành phần	Công nghệ / Thư viện	Vai trò
Nền tảng & Framework	ASP.NET Core Web API (.NET 10)	Xây dựng RESTful API, xử lý routing qua Controller, middleware pipeline.
ORM & Cơ sở dữ liệu	Entity Framework Core 10 + Npgsql.EntityFrameworkCore.PostgreSQL	Mapping đối tượng – quan hệ (Object-Relational Mapping), quản lý migration, truy vấn LINQ tới PostgreSQL.
Xác thực & Phân quyền	Microsoft.AspNetCore.Authentication.JwtBearer	Sinh và xác thực JSON Web Token, phân quyền theo Role (Admin/Staff/Customer).
Mã hóa mật khẩu	BCrypt.Net-Next	Băm một chiều mật khẩu người dùng trước khi lưu vào CSDL.
Đăng nhập Google	Google.Apis.Auth	Xác minh Google ID Token gửi từ Frontend khi người dùng đăng nhập bằng Google OAuth.
Tài liệu API	Swashbuckle.AspNetCore (Swagger / OpenAPI)	Tự sinh giao diện thử nghiệm API, chỉ bật ở môi trường Development.
Xuất báo cáo	ClosedXML	Xuất báo cáo doanh thu sang file Excel (.xlsx) cho phân hệ quản trị.
Chuẩn response	ApiResponse<T> (tự xây dựng)	Đóng gói thống nhất success/data/message/error/pagination cho toàn bộ API.
Xử lý lỗi	ExceptionMiddleware (tự xây dựng)	Bắt exception toàn cục, trả về response

tập trung		lỗi chuẩn hóa, tránh rò rỉ stack trace.
-----------	--	---

3.3. Công nghệ phía Frontend

Bảng 10: Công nghệ phía Frontend

Thành phần	Công nghệ / Thư viện	Vai trò
Thư viện UI	React 19	Xây dựng giao diện theo component, quản lý vòng đời và render.
Build tool / Dev server	Vite 8 (@vitejs/plugin-react)	Bundle mã nguồn, Hot Module Replacement khi phát triển, build production tối ưu.
Định tuyến	React Router DOM 7	Điều hướng SPA, tách layout Admin riêng khỏi layout công khai.
Quản lý state toàn cục	Zustand 5	Lưu trạng thái dùng chung (giỏ hàng, phiên đăng nhập...) gọn nhẹ hơn Redux.
Quản lý dữ liệu server	TanStack React Query 5	Cache, đồng bộ và làm mới dữ liệu lấy từ API một cách hiệu quả.
Gọi API	Axios / Fetch wrapper tự viết (client.js)	Tự động gắn JWT, tự động refresh token khi gặp lỗi 401, quản lý header X-Session-Id.
Quản lý Form	React Hook Form 7	Validate và quản lý dữ liệu các form đăng ký/dăng nhập/checkout/quản trị.
Biểu đồ	Recharts 3	Hiển thị biểu đồ doanh thu/thống kê trong Admin Dashboard.
Icon	lucide-react	Bộ icon nhẹ, dùng nhất quán trong toàn giao diện.
Đăng nhập Google	@react-oauth/google	Hiển thị nút đăng nhập Google và lấy ID Token gửi lên Backend.

3.4. Công nghệ hạ tầng & triển khai

Bảng 11: Công nghệ hạ tầng & triển khai

Thành phần	Công nghệ	Vai trò
Quản lý mã nguồn	Git & GitHub	Lưu trữ mã nguồn, quản lý nhánh, là nguồn kích hoạt (trigger) cho CI/CD của cả Render và Vercel.
Đóng gói ứng dụng	Docker (multi-stage build)	Đóng gói Backend thành image độc lập, tách biệt môi trường build (SDK) và môi trường chạy (ASP.NET runtime) để giảm dung lượng image.
Hosting Backend	Render (Web Service – Docker runtime)	Build image từ Dockerfile, chạy container, expose endpoint public, tự healthcheck qua /health.
Hosting Frontend	Vercel (Static / SPA hosting)	Build dự án Vite, deploy bundle tĩnh lên mạng Edge CDN toàn cầu, hỗ trợ Preview Deployment cho từng Pull Request.

Cơ sở dữ liệu	PostgreSQL trên Supabase (managed)	Lưu trữ toàn bộ dữ liệu quan hệ của hệ thống, truy cập qua chuỗi kết nối khai báo trong biến môi trường.
CI/CD	GitHub Integration (Render & Vercel)	Tự động build & deploy lại mỗi khi có commit/push mới lên nhánh main; tạo bản Preview riêng cho mỗi nhánh/PR (Vercel).

CHƯƠNG 4: THIẾT KẾ HỆ THỐNG

4.1. Kiến trúc xử lý request tổng thể

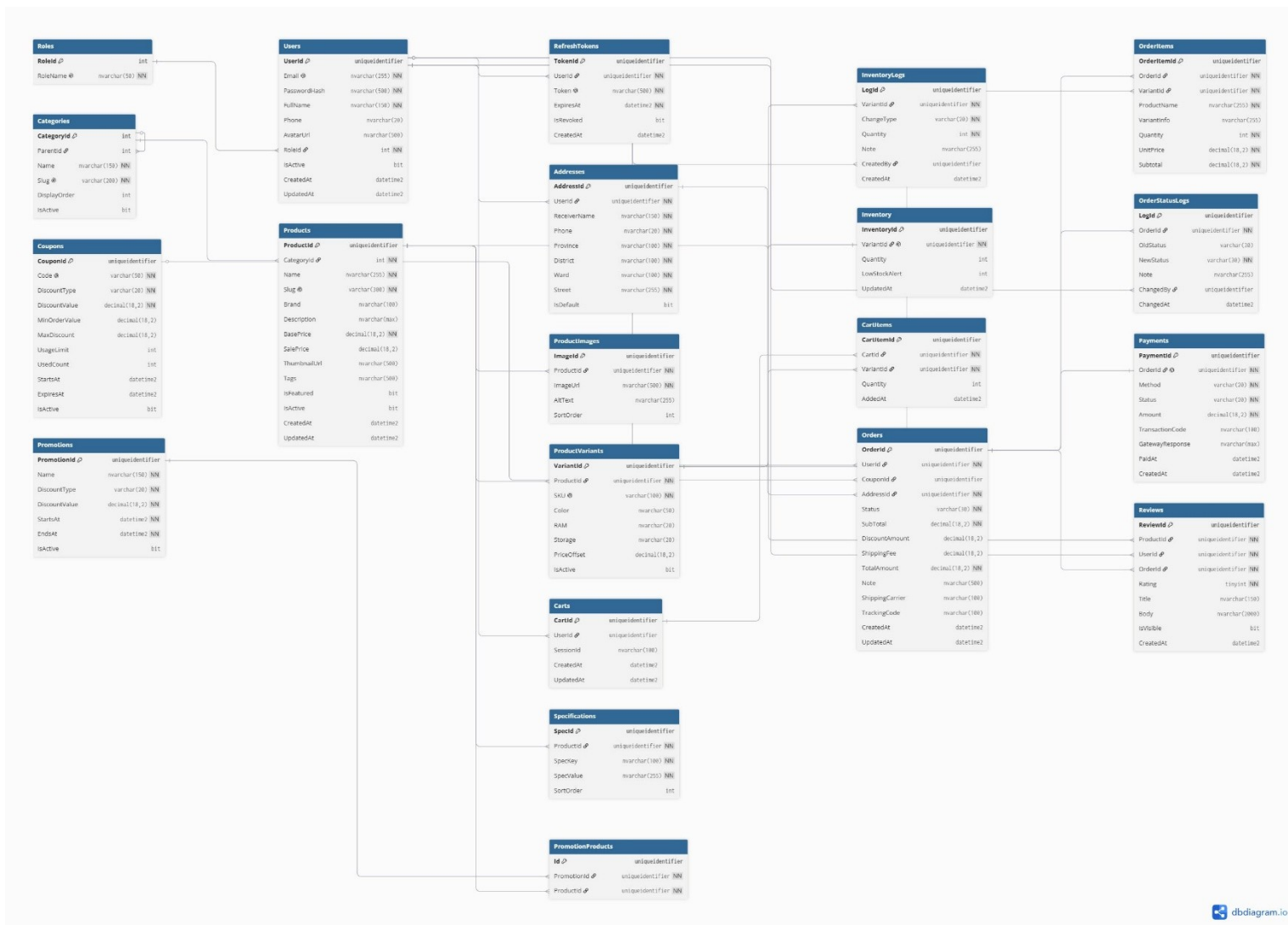
Khi người dùng truy cập website, trình duyệt tải ứng dụng React (đã được build và phân phối qua Vercel). Mọi tương tác cần dữ liệu động (xem sản phẩm, đăng nhập, đặt hàng...) sẽ được Frontend gọi tới Backend API (đang chạy trên Render) thông qua HTTPS. Quá trình xử lý một request tại Backend đi qua pipeline middleware theo thứ tự cố định, được khai báo trong Program.cs:

1. Swagger / Swagger UI — chỉ được kích hoạt khi ASPNETCORE_ENVIRONMENT = Development (không bật ở môi trường Production trên Render, đảm bảo không lộ tài liệu API ra ngoài, nhưng với local để enviroment ở development để dễ chỉnh sửa).
2. ExceptionMiddleware — bắt toàn bộ exception phát sinh ở các bước xử lý sau, trả về ApiResponse lỗi chuẩn hóa.
3. ForwardedHeaders — đọc header X-Forwarded-For / X-Forwarded-Proto do reverse proxy của Render gắn vào, giúp ASP.NET Core nhận đúng giao thức/IP gốc của client.
4. HTTPS Redirection — chỉ áp dụng khi không ở Production (Render đã tự xử lý TLS ở lớp proxy nên Backend không cần redirect lại).
5. CORS (policy "Vite") — chỉ cho phép các origin được khai báo trong biến môi trường Cors:AllowedOrigins (gồm domain Frontend trên Vercel).
6. Authentication — xác thực JWT Bearer Token nếu có.
7. Authorization — kiểm tra Role của user so với yêu cầu của endpoint ([Authorize(Roles = "Admin,Staff")]).
8. MapControllers — định tuyến tới Controller/Action tương ứng, gọi DbContext để truy vấn PostgreSQL và trả kết quả JSON.

4.2. Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu được thiết kế theo mô hình quan hệ chuẩn hóa (chuẩn 3NF ở phần lớn các bảng), bao gồm 21 thực thể chia thành năm nhóm nghiệp vụ: Người dùng/Xác thực, Danh mục/Sản phẩm, Giỏ hàng, Đơn hàng/Thanh toán và Khuyến mãi/Kho/Đánh giá. Toàn bộ thực thể, thuộc tính, kiểu dữ liệu, khóa chính, khóa ngoại và ràng buộc duy nhất (giá trị in đậm "NN" = NOT NULL) được mô tả trực quan trong sơ đồ thực thể liên kết (ERD) ở Hình 1 dưới đây.

Hình 1 — Sơ đồ thực thể liên kết (ERD) cơ sở dữ liệu hệ thống TechShop



4.2.1. Nhóm Người dùng & Xác thực

Bảng 12: Nhóm Người dùng & Xác thực

Thực thể	Mục đích	Trường / chỉ mục quan trọng
Roles	Danh mục vai trò hệ thống: Admin, Staff, Customer.	RoleId (PK); RoleName (unique).
Users	Tài khoản người dùng.	UserId (PK, uniqueidentifier); Email (unique); PasswordHash; RoleId (FK → Roles); IsActive (soft-delete).
RefreshTokens	Token dài hạn dùng để cấp lại accessToken.	TokenId (PK); UserId (FK → Users); Token; ExpiresAt; IsRevoked.
Addresses	Địa chỉ nhận hàng của người dùng, có thể đặt nhiều địa chỉ.	AddressId (PK); UserId (FK → Users); IsDefault.

4.2.2. Nhóm Danh mục & Sản phẩm

Bảng 13: Nhóm Danh mục & Sản phẩm

Thực thể	Mục đích	Trường / chỉ mục quan trọng
Categories	Danh mục sản phẩm, hỗ trợ cấu trúc cha/con.	CategoryId (PK, int); ParentId (self FK); Slug (unique).
Products	Sản phẩm chính.	ProductId (PK); CategoryId (FK); Slug (unique); BasePrice, SalePrice (decimal 18,2); IsFeatured/IsActive.
ProductImages	Ảnh phụ của sản phẩm (nhiều ảnh / sản phẩm).	ImageId (PK); ProductId (FK); SortOrder.
ProductVariants	Biến thể bán theo SKU (màu/RAM/dung lượng) với giá lệch (PriceOffset).	VariantId (PK); ProductId (FK); SKU (unique).
Specifications	Thông số kỹ thuật dạng key-value gắn với sản phẩm.	SpecId (PK); ProductId (FK); SpecKey/SpecValue.

4.2.3. Nhóm Giỏ hàng

Bảng 14: Nhóm Giỏ hàng

Thực thể	Mục đích	Trường / chỉ mục quan trọng
Carts	Giỏ hàng, có thể gắn với UserId (đã đăng nhập) hoặc SessionId (khách).	CartId (PK); UserId (FK, nullable); SessionId.
CartItems	Từng dòng sản phẩm (theo biến thể) trong giỏ.	CartItemId (PK); CartId (FK, cascade delete); VariantId (FK); Quantity.
Coupons	Mã giảm giá áp dụng cho giỏ hàng/đơn hàng.	CouponId (PK); Code (unique); DiscountType/DiscountValue; UsageLimit/UsedCount; StartsAt/ExpiresAt.

4.2.4. Nhóm Đơn hàng & Thanh toán

Bảng 15: Nhóm Đơn hàng & Thanh toán

Thực thể	Mục đích	Trường / chỉ mục quan trọng
Orders	Đơn hàng — chốt thông tin địa chỉ, tổng tiền tại thời điểm đặt.	OrderId (PK); UserId (FK); AddressId (FK); CouponId (FK, nullable); Status; TotalAmount.
OrderItems	Snapshot từng sản phẩm trong đơn (giữ nguyên tên/giá tại thời điểm mua dù sản phẩm gốc thay đổi sau đó).	OrderItemId (PK); OrderId (FK, cascade delete); VariantId (FK); UnitPrice/Subtotal.
OrderStatusLogs	Lịch sử thay đổi trạng thái đơn hàng — phục vụ truy vết/audit.	LogId (PK); OrderId (FK, cascade delete); OldStatus/NewStatus; ChangedBy (FK → Users).
Payments	Thông tin thanh toán gắn 1-1 với đơn hàng.	PaymentId (PK); OrderId (FK, unique); Method; Status; TransactionCode; PaidAt.

4.2.5. Nhóm Khuyến mãi, Kho & Đánh giá

Bảng 16: Nhóm Khuyến mãi, Kho & Đánh giá

Thực thể	Mục đích	Trường / chỉ mục quan trọng
----------	----------	-----------------------------

Promotions	Chương trình khuyến mãi theo khoảng thời gian.	PromotionId (PK); DiscountType/DiscountValue; StartsAt/EndsAt; IsActive.
PromotionProducts	Bảng nối nhiều-nhiều giữa Promotion và Product.	Id (PK); PromotionId (FK); ProductId (FK).
Inventory	Tồn kho hiện tại theo từng biến thể (1-1 với ProductVariant).	InventoryId (PK); VariantId (FK, unique); Quantity; LowStockAlert.
InventoryLogs	Lịch sử nhập/xuất/điều chỉnh/bán/hoàn kho.	LogId (PK); VariantId (FK); ChangeType; Quantity; CreatedBy (FK → Users).
Reviews	Đánh giá sản phẩm của khách hàng, chỉ hiển thị khi isVisible = true.	ReviewId (PK); ProductId (FK); UserId (FK); OrderId (FK); Rating (tinyint, 1-5).

4.3. Ràng buộc & quy ước thiết kế cơ sở dữ liệu

- Khóa chính của hầu hết thực thể dùng kiểu uniqueidentifie, riêng Roles và Categories dùng int tự tăng.
- Các chỉ mục duy nhất (unique index) quan trọng: Roles.RoleName, Categories.Slug, Users.Email, Coupons.Code, Products.Slug, ProductVariants.SKU, Inventory.VariantId, Payments.OrderId.
- Kiểu số tiền (decimal) mặc định dùng precision (18,2) cho toàn bộ các trường giá/tiền.
- Quy tắc xóa: phần lớn khóa ngoại dùng Restrict (không cho xóa thực thể cha khi còn thực thể con tham chiếu); cascade delete chỉ mở cho ba quan hệ: Cart → CartItems, Order → OrderItems và Order → OrderStatusLogs.
- Hệ thống ưu tiên xóa mềm (qua cờ IsActive) đối với Product, User và Promotion thay vì xóa cứng, nhằm bảo toàn dữ liệu lịch sử cho các đơn hàng/báo cáo đã phát sinh trước đó.

4.4. Thiết kế API & chuẩn hóa response

Toàn bộ API được thiết kế theo phong cách RESTful, sử dụng danh từ số nhiều cho resource (/api/products, /api/orders...), phương thức HTTP thể hiện hành động (GET/POST/PUT/PATCH/DELETE), và trả về dữ liệu JSON theo một cấu trúc bao bọc thống nhất là ApiResponse<T>:

Bảng 17: Thiết kế API & chuẩn hóa response

Trường	Kiểu	Ý nghĩa
success	boolean	true nếu request xử lý thành công, false nếu có lỗi.
data	object array null	Dữ liệu trả về (chi tiết sản phẩm, danh sách đơn hàng...).
message	string	Thông báo mô tả kết quả, hiển thị trực tiếp được cho người dùng.
error	string null	Mã lỗi dạng định danh (ví dụ NOT_FOUND, EMAIL_EXISTS) để Frontend xử lý logic riêng nếu cần.
pagination	object null	Chỉ xuất hiện ở API trả danh sách có phân trang: { page, pageSize, total }.

Đối với xác thực, các request cần đăng nhập phải gửi kèm header `Authorization: Bearer <accessToken>`. Đối với giỏ hàng khách (chưa đăng nhập), Backend tự sinh `SessionId` và trả về qua response header `X-Session-Id`; Frontend có trách nhiệm lưu lại và gửi kèm ở các request tiếp theo để duy trì giỏ hàng.

4.5. Thiết kế bảo mật

- Mã hóa mật khẩu: mật khẩu người dùng không bao giờ được lưu ở dạng plaintext; toàn bộ được băm một chiều bằng thuật toán BCrypt (qua thư viện BCrypt.Net-Next) trước khi ghi vào cột PasswordHash.
- Xác thực JWT hai lớp: accessToken có thời hạn ngắn (15 phút, cấu hình qua `Jwt:AccessTokenExpiry`) giảm thiểu rủi ro nếu bị đánh cắp; refreshToken có thời hạn dài hơn (7 ngày, `Jwt:RefreshTokenExpiry`) được lưu trong bảng RefreshTokens kèm cờ IsRevoked để có thể chủ động vô hiệu hóa khi đăng xuất.
- Phân quyền theo Role: claim Role trong JWT (Admin/Staff/Customer) được dùng trong thuộc tính `[Authorize(Roles = "...")]` gắn trên từng Controller/Action để giới hạn truy cập theo đúng vai trò.
- Bảo vệ callback thanh toán: URL callback của VNPay/MoMo/VietQR/ZaloPay (mock) được ký bằng HMAC-SHA256 với khóa bí mật `Payment:CallbackSecret`; khi nhận callback, Backend xác minh lại chữ ký bằng phương pháp so sánh thời gian cố định (fixed-time compare) để phòng chống timing attack, tránh trường hợp giả mạo callback đổi trạng thái thanh toán.
- Kiểm soát CORS chặt theo whitelist: chỉ những origin được khai báo rõ trong biến môi trường `Cors:AllowedOrigins` (ví dụ domain Frontend trên Vercel) mới được phép gọi API; đây cũng là điểm cấu hình bắt buộc phải cập nhật mỗi khi domain Frontend thay đổi sau khi deploy.
- Không để lộ thông tin nhạy cảm: JWT Secret, chuỗi kết nối CSDL và CallbackSecret đều được khai báo `sync: false` trong `render.yaml` — nghĩa là giá trị thật chỉ được nhập trực tiếp và lưu an toàn trên Dashboard của Render, không bao giờ commit vào mã nguồn Git.

CHƯƠNG 5: TRIỂN KHAI CHỨC NĂNG (IMPLEMENTATION)

5.1. Cấu trúc thư mục Backend

Mã nguồn Backend được tổ chức theo mô hình phân lớp rõ ràng, thuận tiện cho việc mở rộng và bảo trì:

```
Backend/
├── Controllers/      # Tiếp nhận request, gọi DbContext, trả ApiResponse
│   ├── AuthController.cs
│   ├── ProductsController.cs
│   ├── CategoriesController.cs
│   ├── BrandsController.cs
│   ├── CartController.cs
│   ├── OrdersController.cs
│   ├── PaymentsController.cs
│   ├── PromotionsController.cs
│   ├── ReviewsController.cs
│   ├── UsersController.cs # Hồ sơ & địa chỉ cá nhân (/api/users/me)
│   └── Admin/        # Các controller chỉ dành cho Admin/Staff
│       ├── AdminOrdersController.cs
│       ├── AdminInventoryController.cs
│       ├── AdminReportsController.cs
│       └── AdminUsersController.cs
├── Data/
│   ├── AppDbContext.cs # Khai báo DbSet + cấu hình quan hệ/ràng buộc
│   └── DbSeeder.cs      # Sinh dữ liệu mẫu khi ứng dụng khởi động
├── DTOs/              # Request/Response model, tách riêng theo từng nhóm nghiệp vụ
├── Models/            # Entity ánh xạ tới bảng CSDL
├── Services/
│   ├── EmailService.cs # Hiện là mock — log ra console
│   └── PaymentGatewayService.cs # Sinh URL thanh toán mock + ký/xác minh HMAC
├── Middleware/
│   └── ExceptionMiddleware.cs # Xử lý lỗi toàn cục
├── Migrations/        # Lịch sử migration EF Core (InitialPostgresSchema, AddGoogleId...)
├── Program.cs         # Khởi tạo, đăng ký service, cấu hình middleware pipeline
└── api.csproj         # Khai báo target framework & package reference
```

5.2. Cấu trúc thư mục Frontend

```
Frontend/
├── src/
│   ├── api/
│   │   ├── client.js # Wrapper fetch: gắn JWT, auto refresh-token, X-Session-Id
│   │   ├── adminApi.js # Hàm gọi API dành cho phân hệ quản trị
│   │   └── mappers.js # Chuẩn hóa dữ liệu API về định dạng UI cần
│   ├── context/      # AuthContext, CartContext, ProductContext (React Context)
│   ├── components/   # Header, Footer, ProductCard, Cart, Checkout, Auth, FilterBar...
│   ├── pages/        # HomePage, LaptopPage, PhonePage, AccessoryPage,
│   │                 # ProductDetailPage, CartPage, CheckoutPage, AccountPage...
│   ├── admin/        # AdminDashboard, AdminProducts, AdminOrders, AdminInventory,
│   │                 # AdminUsers, AdminPromotions, AdminReports
│   ├── assets/       # Ảnh, icon, logo các hãng
│   ├── App.jsx       # Khai báo toàn bộ Route (React Router)
│   ├── main.jsx      # Điểm khởi chạy ứng dụng React
│   └── index.html
└──
```

```
└─ vite.config.js
└─ package.json
```

5.3. Các chức năng chính đã triển khai

Bảng 18: Các chức năng chính đã triển khai

Module	Chức năng đã hoàn thiện
Xác thực (Auth)	Đăng ký, đăng nhập email/mật khẩu, đăng nhập Google OAuth, refresh token tự động (interceptor 401), đăng xuất, modal Auth dùng chung toàn site.
Danh mục & Sản phẩm	Trang chủ, trang Laptop/Điện thoại/Phụ kiện, trang theo hãng (BrandSeriesPage), trang chi tiết sản phẩm với bảng thông số kỹ thuật, bộ lọc và sắp xếp.
Giỏ hàng	Thêm/sửa/xóa sản phẩm, hỗ trợ giỏ hàng khách lẫn giỏ hàng người dùng đã đăng nhập, áp dụng mã giảm giá, hiển thị tạm tính/giảm giá/tổng tiền.
Thanh toán & Đặt hàng	Trang Checkout thu thập địa chỉ giao hàng, chọn phương thức COD/VNPay/MoMo/VietQR/ZaloPay (mock), tạo đơn hàng, xem lịch sử đơn, hủy đơn.
Tài khoản	Trang quản lý hồ sơ cá nhân, địa chỉ giao hàng, lịch sử đơn hàng (AccountPage).
Quản trị — Dashboard	Tổng quan số liệu (doanh thu, đơn hàng, người dùng) kèm biểu đồ Recharts.
Quản trị — Sản phẩm	CRUD sản phẩm, quản lý ảnh, biến thể, thông số kỹ thuật.
Quản trị — Đơn hàng	Xem danh sách, lọc theo trạng thái, cập nhật trạng thái & mã vận đơn.
Quản trị — Kho hàng	Xem tồn kho theo biến thể, nhập/xuất/điều chỉnh kho, xem lịch sử log kho, cảnh báo tồn kho thấp.
Quản trị — Khuyến mãi	Tạo/sửa/tắt chương trình khuyến mãi, gán danh sách sản phẩm áp dụng.
Quản trị — Người dùng	Tạo/sửa người dùng, khóa/mở tài khoản, đổi mật khẩu, xóa mềm.
Quản trị — Báo cáo	Báo cáo doanh thu theo ngày/tháng, top sản phẩm bán chạy, sản phẩm tồn kho thấp, xuất Excel.

5.4. Danh sách đầy đủ API endpoint

Phần này tổng hợp toàn bộ endpoint hiện có của Backend, được nhóm theo từng Controller, phục vụ tra cứu khi tích hợp Frontend hoặc kiểm thử thủ công.

5.4.1. AuthController — /api/auth

Bảng 19: AuthController — /api/auth

Method	Endpoint	Quyền	Mục đích
POST	/api/auth/register	Public	Đăng ký tài khoản Customer mới.
POST	/api/auth/login	Public	Đăng nhập email/mật khẩu, trả accessToken + refreshToken.
POST	/api/auth/google-login	Public	Đăng nhập bằng Google ID Token.
POST	/api/auth/refresh	Public	Cấp accessToken mới từ refreshToken còn hiệu lực.
POST	/api/auth/logout	Public	Revoke refreshToken hiện tại.
POST	/api/auth/change-password	Customer	Đổi mật khẩu khi đã đăng nhập (cần mật khẩu cũ).
POST	/api/auth/forgot-	Public	Quên mật khẩu: sinh mật khẩu mới ngẫu nhiên

	password		và gửi qua email thật bằng Brevo Transactional Email API.
--	----------	--	---

5.4.2. ProductsController — /api/products

Bảng 20: ProductsController — /api/products

Method	Endpoint	Quyền	Mục đích
GET	/api/products	Public	Danh sách sản phẩm: lọc category/brand/giá, sắp xếp, phân trang.
GET	/api/products/{slug}	Public	Chi tiết sản phẩm theo Slug (kèm ảnh, biến thể, thông số, rating).
POST	/api/products	Admin, Staff	Tạo sản phẩm mới.
PUT	/api/products/{id}	Admin, Staff	Cập nhật thông tin sản phẩm.
DELETE	/api/products/{id}	Admin	Xóa mềm sản phẩm (IsActive = false).
POST	/api/products/{id}/images	Admin, Staff	Tải ảnh phụ cho sản phẩm.
GET	/api/products/{id}/reviews	Public	Lấy danh sách đánh giá đang hiển thị của sản phẩm.

5.4.3. CategoriesController & BrandsController

Bảng 21: CategoriesController & BrandsController

Method	Endpoint	Quyền	Mục đích
GET	/api/categories	Public	Lấy cây danh mục cha/con.
GET	/api/brands	Public	Lấy danh sách hãng distinct từ sản phẩm đang active.

5.4.4. CartController — /api/cart

Bảng 22: CartController — /api/cart

Method	Endpoint	Quyền	Mục đích
GET	/api/cart	Optional	Lấy hoặc tạo giỏ hàng (theo UserId hoặc SessionId).
POST	/api/cart/items	Optional	Thêm một biến thể sản phẩm vào giỏ.
PUT	/api/cart/items/{id}	Optional	Cập nhật số lượng của một item trong giỏ.
DELETE	/api/cart/items/{id}	Optional	Xóa item khỏi giỏ.
POST	/api/cart/apply-coupon	Optional	Áp dụng mã giảm giá cho giỏ hàng hiện tại.

5.4.5. OrdersController — /api/orders (yêu cầu đăng nhập)

Bảng 23: OrdersController — /api/orders (yêu cầu đăng nhập)

Method	Endpoint	Quyền	Mục đích
POST	/api/orders	Customer	Tạo đơn hàng từ giỏ hàng hiện tại.
GET	/api/orders	Customer	Lấy lịch sử đơn hàng của người dùng đang đăng nhập.
GET	/api/orders/{id}	Customer	Xem chi tiết một đơn hàng.
PATCH	/api/orders/{id}/cancel	Customer	Hủy đơn hàng (nếu còn hợp lệ).
PATCH	/api/orders/{id}/address	Customer	Cập nhật địa chỉ giao hàng của đơn (khi đơn còn cho phép sửa).

5.4.6. PaymentsController — /api/payments

Bảng 24: PaymentsController — /api/payments

Method	Endpoint	Quyền	Mục đích
POST	/api/payments/vnpay/create	Customer	Tạo URL thanh toán VNPay (mock).
POST	/api/payments/momo/create	Customer	Tạo URL thanh toán MoMo (mock).
POST	/api/payments/vnpay/callback	Public	Callback VNPay (mock), xác minh chữ ký HMAC.
POST	/api/payments/momo/callback	Public	Callback MoMo (mock), xác minh chữ ký HMAC.
POST	/api/payments/qr/create	Customer	Tạo mã/URL thanh toán VietQR (mock).
POST	/api/payments/zalopay/create	Customer	Tạo URL thanh toán ZaloPay (mock).
POST	/api/payments/qr/callback	Public	Callback VietQR (mock), xác minh chữ ký HMAC.
POST	/api/payments/zalopay/callback	Public	Callback ZaloPay (mock), xác minh chữ ký HMAC.

5.4.7. PromotionsController — /api/promotions

Bảng 25: PromotionsController — /api/promotions

Method	Endpoint	Quyền	Mục đích
GET	/api/promotions/active	Public	Lấy chương trình khuyến mãi đang còn hiệu lực.
GET	/api/promotions	Admin, Staff	Danh sách toàn bộ chương trình khuyến mãi.
GET	/api/promotions/{id}	Admin, Staff	Chi tiết một chương trình khuyến mãi.
POST	/api/promotions	Admin, Staff	Tạo chương trình khuyến mãi mới.
PUT	/api/promotions/{id}	Admin, Staff	Cập nhật chương trình khuyến mãi.
DELETE	/api/promotions/{id}	Admin, Staff	Tắt chương trình khuyến mãi.

5.4.8. ReviewsController

Bảng 26: ReviewsController

Method	Endpoint	Quyền	Mục đích
POST	/api/reviews	Customer	Gửi đánh giá cho sản phẩm thuộc đơn hàng đã Completed.

5.4.9. AdminOrdersController — /api/admin/orders

Bảng 27: AdminOrdersController — /api/admin/orders

Method	Endpoint	Quyền	Mục đích
GET	/api/admin/orders	Admin, Staff	Danh sách toàn bộ đơn hàng, lọc theo trạng thái, có phân trang.
PATCH	/api/admin/orders/{id}/status	Admin, Staff	Cập nhật trạng thái đơn hàng (ghi OrderStatusLog).
PUT	/api/admin/orders/{id}/tracking	Admin, Staff	Cập nhật mã vận đơn.

5.4.10. AdminInventoryController — /api/admin/inventory

Bảng 28: AdminInventoryController — /api/admin/inventory

Method	Endpoint	Quyền	Mục đích
GET	/api/admin/inventory	Admin, Staff	Danh sách tồn kho theo biến thể.
GET	/api/admin/inventory/logs	Admin, Staff	Lịch sử log nhập/xuất/điều chỉnh kho.
POST	/api/admin/inventory/import	Admin, Staff	Nhập kho (ChangeType = Import).
POST	/api/admin/inventory/export	Admin, Staff	Xuất kho (ChangeType = Export).
PATCH	/api/admin/inventory/adjust	Admin, Staff	Điều chỉnh số lượng tồn kho (ChangeType = Adjust).

5.4.11. AdminReportsController — /api/admin/reports**Bảng 29: AdminReportsController — /api/admin/reports**

Method	Endpoint	Quyền	Mục đích
GET	/api/admin/reports/revenue	Admin, Staff	Báo cáo doanh thu theo khoảng thời gian, gom nhóm theo day/month.
GET	/api/admin/reports/top-products	Admin, Staff	Top sản phẩm bán chạy.
GET	/api/admin/reports/low-stock	Admin, Staff	Danh sách sản phẩm/biến thể tồn kho thấp.
GET	/api/admin/reports/revenue/export	Admin, Staff	Xuất báo cáo doanh thu ra file Excel (.xlsx) bằng ClosedXML.

5.4.12. AdminUsersController — /api/admin/users**Bảng 30: AdminUsersController — /api/admin/users**

Method	Endpoint	Quyền	Mục đích
GET	/api/admin/users	Admin	Danh sách người dùng, hỗ trợ tìm kiếm/lọc/phân trang.
GET	/api/admin/users/{id}	Admin	Chi tiết một người dùng.
POST	/api/admin/users	Admin	Tạo người dùng mới (có thể tạo Staff/Admin).
PUT	/api/admin/users/{id}	Admin	Cập nhật thông tin người dùng.
PATCH	/api/admin/users/{id}/status	Admin	Bật/tắt trạng thái hoạt động tài khoản.
PATCH	/api/admin/users/{id}/password	Admin	Đổi mật khẩu cho người dùng.
DELETE	/api/admin/users/{id}	Admin	Xóa mềm người dùng (IsActive = false).

5.4.13. UsersController — /api/users (tài khoản cá nhân)**Bảng 31: UsersController — /api/users (tài khoản cá nhân)**

Method	Endpoint	Quyền	Mục đích
GET	/api/users/me	Customer	Lấy hồ sơ cá nhân của người dùng đang đăng nhập.
PUT	/api/users/me	Customer	Cập nhật hồ sơ cá nhân (họ tên, số điện thoại...).
GET	/api/users/me/addresses	Customer	Lấy danh sách địa chỉ nhận hàng của người dùng.
POST	/api/users/me/addresses	Customer	Thêm địa chỉ nhận hàng mới.
PUT	/api/users/me/addresses/{id}	Customer	Cập nhật một địa chỉ nhận hàng.

PATCH	/api/users/me/addresses/{id}/default	Customer	Đặt một địa chỉ làm mặc định.
DELETE	/api/users/me/addresses/{id}	Customer	Xóa một địa chỉ nhận hàng.

Tổng cộng hệ thống hiện cung cấp 67 endpoint REST API, được tổ chức nhất quán theo chuẩn ApiResponse<T> đã trình bày ở mục 4.4, cùng endpoint GET /health phục vụ riêng cho cơ chế Health Check khi triển khai trên Render.

5.5. Dữ liệu mẫu (Seed data)

Lớp DbSeeder được gọi tự động mỗi khi Backend khởi động , được sử dụng trong lúc phát triển backend, nhằm kiểm thử dữ liệu. Sau khi triển khai, được xóa bỏ nhằm tránh gây xung đột dữ liệu với data chính.

CHƯƠNG 6: TRIỂN KHAI HỆ THỐNG LÊN RENDER VÀ VERCEL

6.1. Tổng quan chiến lược triển khai

Vì Frontend và Backend được phát triển hoàn toàn độc lập (giao tiếp qua REST API), nhóm lựa chọn chiến lược triển khai tách rời, sử dụng hai nền tảng cloud chuyên biệt cho từng loại workload, đều hỗ trợ gói miễn phí phù hợp với quy mô đồ án:

- Render — dùng để triển khai Backend (ASP.NET Core Web API). Render hỗ trợ chạy Web Service trực tiếp từ Dockerfile, rất phù hợp với một ứng dụng .NET cần môi trường runtime đầy đủ, có thể truy cập cơ sở dữ liệu PostgreSQL liên tục.
- Vercel — dùng để triển khai Frontend (ứng dụng React/Vite). Vercel chuyên tối ưu cho các ứng dụng tĩnh/SPA, build cực nhanh, phân phối qua mạng Edge CDN toàn cầu giúp người dùng cuối tải trang nhanh hơn, và tự tạo URL Preview cho từng nhánh/Pull Request.

Việc tách triển khai theo đúng bản chất của từng thành phần (Backend cần server chạy liên tục xử lý logic + kết nối CSDL; Frontend chỉ là tập tệp tĩnh sau khi build) là một thực hành phổ biến trong kiến trúc web hiện đại (Decoupled/Headless architecture), giúp mỗi phần có thể scale, cập nhật và rollback độc lập với phần còn lại.

Tại thời điểm thực hiện báo cáo, Backend của hệ thống đã được triển khai thành công và đang hoạt động công khai tại địa chỉ:

<https://techshop-backend-8sfu.onrender.com>

Địa chỉ này được Frontend sử dụng làm giá trị mặc định (fallback) trong tệp `src/api/client.js`, cho phép ứng dụng React hoạt động đúng ngay cả khi chưa kịp khai báo biến môi trường `VITE_API_URL` ở môi trường build.

6.2. Triển khai Backend lên Render

6.2.1. Các tệp cấu hình triển khai có sẵn trong repository

Repository đã có sẵn hai tệp cấu hình then chốt cho việc triển khai Backend bằng Docker trên Render:

Dockerfile (multi-stage build):

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src

COPY Backend/api.csproj Backend/
RUN dotnet restore Backend/api.csproj

COPY Backend/ Backend/
RUN dotnet publish Backend/api.csproj -c Release -o /app/publish --no-restore

FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runtime
WORKDIR /app
```

```
RUN apt-get update \
  && apt-get install -y --no-install-recommends libgssapi-krb5-2 \
  && rm -rf /var/lib/apt/lists/*

ENV ASPNETCORE_ENVIRONMENT=Production
EXPOSE 10000

COPY --from=build /app/publish .

ENTRYPOINT ["sh", "-c", "ASPNETCORE_HTTP_PORTS=${PORT:-10000} dotnet api.dll"]
```

Dockerfile được thiết kế theo mô hình multi-stage build gồm hai giai đoạn:

1. Giai đoạn build: dùng image đầy đủ mcr.microsoft.com/dotnet/sdk:10.0 để restore package và publish ứng dụng ra thư mục /app/publish ở chế độ Release.
2. Giai đoạn runtime: chỉ dùng image mcr.microsoft.com/dotnet/aspnet:10.0 (nhẹ hơn nhiều so với image SDK) để chạy ứng dụng đã publish, giúp giảm đáng kể dung lượng image cuối cùng. Gói libgssapi-krb5-2 được cài thêm để hỗ trợ thư viện Npgsql trong một số cơ chế xác thực Kerberos của PostgreSQL.
3. Dòng ENTRYPOINT đọc biến môi trường PORT do Render cấp phát động (mặc định 10000 nếu không có) và gán vào ASPNETCORE_HTTP_PORTS trước khi chạy dotnet api.dll — đây là điểm bắt buộc phải có vì Render không cho phép cấu hình cứng cổng lắng nghe trong image.

render.yaml (Render Blueprint — Infrastructure as Code):

```
services:
- type: web
  name: techshop-backend
  runtime: docker
  plan: free
  region: singapore
  dockerfilePath: ./Dockerfile
  dockerContext: .
  healthCheckPath: /health
  envVars:
  - key: PORT
    value: 10000
  - key: ASPNETCORE_ENVIRONMENT
    value: Production
  - key: ConnectionStrings__DefaultConnection
    sync: false
  - key: Jwt__Secret
    sync: false
  - key: Jwt__Issuer
    value: TechShop
  - key: Jwt__Audience
    value: TechShopClient
  - key: Jwt__AccessTokenExpiry
    value: 15
  - key: Jwt__RefreshTokenExpiry
    value: 7
  - key: Payment__CallbackSecret
```

```
sync: false
- key: Cors__AllowedOrigins
sync: false
```

Tệp render.yaml cho phép khai báo toàn bộ hạ tầng dưới dạng mã (Blueprint). Khi import repository vào Render, nền tảng sẽ tự đọc tệp này để biết cách build, region triển khai (singapore — gần người dùng Việt Nam, giảm độ trễ), gói dịch vụ (free) và danh sách biến môi trường cần thiết. Những biến có sync: false là biến nhạy cảm — Render sẽ không tự sinh giá trị, mà yêu cầu người triển khai nhập trực tiếp trên Dashboard, đảm bảo không có secret nào bị lưu trong Git.

6.2.2. Các bước triển khai chi tiết trên Render

1. Đăng nhập/Tạo tài khoản tại render.com, liên kết với tài khoản GitHub của nhóm để Render có quyền đọc repository.
2. Tạo thủ công bằng New → Web Service, chọn Runtime là Docker, Dockerfile Path là ./Dockerfile, Docker Build Context Directory là . (thư mục gốc repo, vì Dockerfile cần COPY cả Backend/ từ gốc).
3. Chọn Region: Singapore để giảm độ trễ cho người dùng khu vực Đông Nam Á, và Instance Type: Free.
4. Khai báo Health Check Path là /health — endpoint này đã được định nghĩa sẵn trong Program.cs (app.MapGet("/health", ...)) và trả về { status: "ok" }; Render dùng endpoint này để theo dõi tình trạng container và tự khởi động lại nếu phát hiện service không phản hồi.
5. Tại tab Environment, khai báo đầy đủ các biến môi trường bắt buộc (xem Bảng 33) — đặc biệt là ConnectionStrings__DefaultConnection (chuỗi kết nối tới PostgreSQL), Jwt__Secret (khóa bí mật để ký JWT, nên dùng chuỗi ngẫu nhiên đủ dài, ví dụ sinh bằng openssl rand -base64 48) và Payment__CallbackSecret.
6. Nhấn Create Web Service / Deploy. Render sẽ tự clone repository, build Docker image theo Dockerfile, sau đó chạy container và expose ra một domain dạng https://<service-name>.onrender.com.
7. Theo dõi tab Logs trong quá trình build/deploy để phát hiện lỗi (ví dụ thiếu biến môi trường, lỗi kết nối CSDL, migration chưa áp dụng...).
8. Sau khi service chuyển trạng thái Live, kiểm tra bằng cách truy cập https://<service-name>.onrender.com/health — nếu trả về JSON { "status": "ok" } tức là Backend đã hoạt động đúng.

6.2.3. Bảng biến môi trường Backend trên Render

Bảng 32: Biến môi trường Backend trên Render

Biến môi trường	Vai trò	Ví dụ giá trị
PORT	Cổng nội bộ container lắng nghe (Render tự	10000

	gán động, ENTRYPOINT đọc biến này).	
ASPNETCORE_ENVIRONMENT	Xác định môi trường chạy; đặt Production để tắt Swagger và bật HTTPS redirection do proxy xử lý.	Production
ConnectionStrings__DefaultConnection	Chuỗi kết nối Npgsql tới CSDL PostgreSQL trên Supabase (managed PostgreSQL, dùng Ssl Mode=Require).	Host=...;Port=5432;Database=techshop;Username=...;Password=...;Ssl Mode=Require
Jwt__Secret	Khóa bí mật đối xứng dùng để ký/xác thực JWT.	(chuỗi ngẫu nhiên ≥ 32 ký tự)
Jwt__Issuer / Jwt__Audience	Định danh Issuer/Audience của token, dùng để xác thực chiều phát hành token.	TechShop / TechShopClient
Jwt__AccessTokenExpiry	Thời hạn accessToken (phút).	15
Jwt__RefreshTokenExpiry	Thời hạn refreshToken (ngày).	7
Payment__CallbackSecret	Khóa bí mật ký HMAC-SHA256 cho URL callback thanh toán mock.	(chuỗi ngẫu nhiên)
Cors__AllowedOrigins	Danh sách	https://techshop-55gp.vercel.app,http://localhost:5173

	origin Frontend được phép gọi API, phân tách bằng dấu phẩy.	
--	--	--

6.2.4. Cấu hình cơ sở dữ liệu PostgreSQL trên Supabase

Cơ sở dữ liệu production của hệ thống được đặt trên Supabase — nền tảng cung cấp PostgreSQL được quản lý miễn phí, kèm sẵn connection pooler, sao lưu tự động và bảng điều khiển trực quan. Backend trên Render kết nối tới CSDL này qua biến môi trường; chỉ cần lấy được chuỗi kết nối đúng định dạng Npgsql để khai báo vào ConnectionStrings__DefaultConnection. Quy trình khuyến nghị:

1. Tạo một project trên Supabase (supabase.com); Supabase sẽ tự khởi tạo một cơ sở dữ liệu PostgreSQL được quản lý cho project.
2. Trong Supabase, vào Project Settings → Database để lấy chuỗi kết nối (ưu tiên Connection Pooler), rồi chuyển sang định dạng key-value mà Npgsql yêu cầu: Host, Port, Database, Username, Password, Ssl Mode=Require.
3. Khai báo chuỗi kết nối vào biến ConnectionStrings__DefaultConnection của Web Service techshop-backend trên Render.
4. Áp dụng migration lên CSDL production bằng lệnh dotnet ef database update với --connection chỉ tới CSDL production (chạy một lần từ máy local hoặc qua một Job riêng), hoặc cấu hình ứng dụng tự động chạy migration khi khởi động.
5. Khi Backend khởi động lần đầu, DbSeeder.Seed() sẽ tự sinh dữ liệu mẫu (role, category, tài khoản admin/customer mẫu, sản phẩm mẫu...) trực tiếp trên CSDL production, sẵn sàng để demo ngay sau khi deploy.

6.2.5. Quy trình build & deploy tự động

- Render liên kết trực tiếp với nhánh main của repository GitHub; mỗi khi có commit mới được push lên main, Render tự động kích hoạt một lượt build mới.
- Quy trình build: Render kéo mã nguồn mới nhất → build Docker image theo đúng Dockerfile (hai giai đoạn build/runtime đã trình bày ở 6.2.1) → nếu build thành công, container cũ được thay thế bằng container mới (zero-downtime deploy ở các gói trả phí; ở gói Free có thể có một khoảng downtime ngắn).
- Có thể theo dõi từng lượt deploy trên Dashboard, xem log build/runtime, và rollback về một bản deploy thành công trước đó chỉ với một cú click nếu bản mới phát sinh lỗi.

6.2.6. Giới hạn của gói Free trên Render và cách xử lý

- Spin down khi không hoạt động: Web Service ở gói Free sẽ tự động “ngủ” sau khoảng 15 phút không có request nào tới. Khi có request đầu tiên sau thời gian ngủ, Render cần thời gian khởi

động lại container, khiến request đó phản hồi chậm hơn đáng kể (có thể từ vài giây đến khoảng 1 phút).

- Giải pháp giảm thiểu cold start (*dự tính*): thiết lập một tác vụ định kỳ gọi định kỳ tới endpoint GET /health mỗi 10–14 phút để giữ service luôn ở trạng thái “thức”; hoặc thông báo trước cho người dùng/giáo viên về độ trễ lần truy cập đầu tiên khi demo.
- Giới hạn giờ build: gói Free có giới hạn số giờ build/tháng; nhóm cần tránh push quá nhiều commit nhỏ liên tục lên main để không làm tiêu hao hạn mức không cần thiết.

6.2.7. Kiểm tra sau khi triển khai

- Gọi GET `https://techshop-backend-8sfu.onrender.com/health`, kỳ vọng nhận về JSON dạng `{ "status": "ok" }`.
- Vì ASPNETCORE_ENVIRONMENT=Production, Swagger UI sẽ không khả dụng ở domain Production (đây là chủ đích thiết kế nhằm tránh lộ tài liệu API công khai); muốn xem Swagger, cần chạy Backend ở môi trường Development (local).
- Thử gọi thử một endpoint public, ví dụ GET /api/products, để xác nhận DbSeeder đã chạy và CSDL production có dữ liệu mẫu.
- Kiểm tra phần Logs trên Render khi gặp lỗi 500 — phần lớn nguyên nhân thường gặp là thiếu/sai biến môi trường (đặc biệt Jwt__Secret hoặc ConnectionStrings__DefaultConnection) hoặc CSDL production chưa được áp dụng migration.

6.3. Triển khai Frontend lên Vercel

6.3.1. Chuẩn bị trước khi triển khai

Frontend được xây dựng bằng Vite, vốn đã hỗ trợ sẵn (zero-config) trên Vercel. Trước khi triển khai cần đảm bảo các điểm sau:

- Script build trong package.json là vite build, sinh ra thư mục dist/ chứa toàn bộ tệp tĩnh production (đã được kiểm tra có sẵn trong cấu hình hiện tại của Frontend/package.json).
- Toàn bộ địa chỉ gọi API trong Frontend phải đọc từ biến môi trường (import.meta.env.VITE_API_URL) thay vì hard-code localhost, đúng như cách client.js đã triển khai (có giá trị fallback là domain Render thật nên ở mục 6.1 để vẫn hoạt động được nếu quên khai báo biến môi trường).
- Vì ứng dụng dùng React Router với chế độ client-side routing (Browser Router) cho các đường dẫn như /laptop, /san-pham/:id, /tai-khoan..., cần cấu hình rewrite toàn bộ route về index.html trên Vercel để tránh lỗi 404 khi người dùng truy cập trực tiếp hoặc tải lại (refresh) một route con.

6.3.2. Các bước triển khai chi tiết trên Vercel

1. Đăng nhập vercel.com bằng tài khoản GitHub, chọn Add New → Project.

2. Chọn import repository PiupiuTenshi/TechWeb-2026. Vì repository chứa cả Backend và Frontend trong cùng một repo (monorepo dạng đơn giản), tại bước cấu hình Project cần khai báo Root Directory là Frontend, để Vercel chỉ build đúng phần ứng dụng React, không động tới mã nguồn .NET.
3. Vercel sẽ tự nhận diện Framework Preset là Vite. Giữ nguyên các lệnh mặc định: Build Command: npm run build (tương đương vite build), Output Directory: dist, Install Command: npm install.
4. Tại bước Environment Variables, khai báo các biến cần thiết cho môi trường Production (xem Bảng 34).
5. Nhấn Deploy. Vercel sẽ cài dependency, chạy lệnh build, sau đó publish thư mục dist/ lên mạng Edge CDN và cấp một domain dạng <https://<project-name>.vercel.app>.
6. Sau khi có domain Frontend chính thức, quay lại Render, cập nhật biến Cors__AllowedOrigins của Backend để bổ sung domain Vercel này vào danh sách được phép gọi API, rồi deploy lại Backend (hoặc Render service sẽ tự nhận biến môi trường mới ngay khi lưu, tùy cấu hình).
7. Truy cập domain Vercel, kiểm thử toàn bộ luồng (đăng nhập, xem sản phẩm, thêm giỏ hàng, đặt hàng) để xác nhận Frontend đã gọi đúng tới Backend trên Render và không còn lỗi CORS.

6.3.3. Bảng biến môi trường Frontend trên Vercel

Bảng 33: Biến môi trường Frontend trên Vercel

Biến môi trường	Vai trò	Ví dụ giá trị
VITE_API_URL	Địa chỉ gốc (base URL) của Backend API mà Frontend sẽ gọi tới.	https://techshop-backend-8sfu.onrender.com
VITE_GOOGLE_CLIENT_ID	Client ID của Google OAuth, dùng để hiển thị và xử lý nút đăng nhập Google.	xxxxxxxxx.apps.googleusercontent.com

Lưu ý: theo quy ước của Vite, chỉ những biến môi trường có tiền tố VITE_ mới được bundler đưa vào mã nguồn phía client (qua import.meta.env); các biến không có tiền tố này sẽ bị bỏ qua hoàn toàn khi build, đây là cơ chế bảo vệ giúp tránh vô tình làm lộ biến môi trường nhạy cảm phía server ra trình duyệt.

6.3.4. Cấu hình SPA Fallback (vercel.json)

Repository hiện đã có sẵn tệp Frontend/vercel.json, giúp đảm bảo các route con của React Router (ví dụ /laptop/asus, /san-pham/iphone-15-pro-max) không bị Vercel trả lỗi 404 khi người dùng truy cập trực tiếp bằng URL hoặc tải lại trang. Nội dung tệp Frontend/vercel.json như sau:

```
{
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

Quy tắc rewrite này yêu cầu Vercel luôn trả về index.html cho mọi đường dẫn không khớp với một tệp tĩnh thật sự trong thư mục dist/ (ảnh, CSS, JS...); sau đó chính React Router (chạy trong trình duyệt) sẽ đọc URL và render đúng trang tương ứng.

6.3.5. Auto-Deploy và Preview Deployment

- Production Deployment: mỗi commit được push lên nhánh main sẽ tự động kích hoạt một lượt build và publish lại lên domain Production chính (<https://<project-name>.vercel.app>).
- Preview Deployment: mỗi khi tạo một nhánh mới hoặc một Pull Request, Vercel tự động build và cấp một domain Preview riêng biệt (dạng <https://<project-name>-<hash>.vercel.app>), cho phép xem trước thay đổi giao diện mà không ảnh hưởng tới bản Production — rất phù hợp để các thành viên trong nhóm review UI trước khi merge.
- Toàn bộ lịch sử Deployment được lưu lại trên Dashboard, có thể xem log build, và rollback về bất kỳ bản deploy trước đó chỉ với một cú click.

6.4. Kết nối Frontend và Backend sau khi triển khai (CORS)

Một trong những lỗi phổ biến nhất khi tách triển khai Frontend/Backend trên hai nền tảng khác nhau là lỗi CORS (Cross-Origin Resource Sharing), do trình duyệt chặn request từ domain Vercel gọi sang domain Render nếu Backend không khai báo rõ origin đó được phép. Quy trình đồng bộ cấu hình hai chiều cần thực hiện:

Bảng 34: Kết nối Frontend và Backend sau khi triển khai (CORS)

Bước	Thực hiện trên	Nội dung
1	Vercel	Lấy domain Production chính thức sau khi deploy Frontend lần đầu, cụ thể là https://techshop-55gp.vercel.app .
2	Render	Cập nhật biến Cors__AllowedOrigins thành danh sách origin được phép, gồm cả domain Vercel Production và (nếu cần) các domain Preview, phân tách bằng dấu phẩy.
3	Render	Lưu lại biến môi trường — Render sẽ tự khởi động lại service để áp dụng cấu hình CORS mới.
4	Vercel	Cập nhật biến VITE_API_URL trỏ đúng domain Backend trên Render, sau đó Redeploy lại Frontend để bundle JS build lại với giá trị môi trường mới (vì Vite inject biến môi trường tại thời điểm build, không phải runtime).

Đây cũng chính là phần “sửa lại cấu hình triển khai” quan trọng nhất khi chuyển từ môi trường phát triển local (Frontend gọi <http://localhost:5000>, không bị chặn CORS vì đã khai báo sẵn trong Program.cs) sang môi trường thật trên Render + Vercel: hai domain hoàn toàn khác nhau, do đó cấu hình CORS và biến môi trường URL API là bắt buộc phải rà soát và cập nhật chính xác.

6.5. Sơ đồ luồng triển khai tổng thể

Toàn bộ quy trình từ lúc lập trình viên đẩy mã nguồn lên GitHub cho tới khi người dùng cuối truy cập được hệ thống diễn ra như sau:

1. Lập trình viên commit và push mã nguồn lên nhánh main của repository PiupiuTenshi/TechWeb-2026 trên GitHub.
2. GitHub gửi webhook thông báo có thay đổi tới cả hai nền tảng đã liên kết: Render và Vercel.
3. Render: kéo mã nguồn mới → build Docker image theo Dockerfile (chỉ build lại phần Backend) → chạy container mới → container expose endpoint tại <https://techshop-backend-8sfu.onrender.com>.
4. Vercel: kéo mã nguồn mới → vào thư mục Frontend → cài dependency → chạy vite build → publish thư mục dist/ lên mạng Edge CDN toàn cầu → cấp/refresh domain Production.
5. Người dùng cuối truy cập domain Vercel; trình duyệt tải về ứng dụng React (tĩnh, phân phối từ Edge gần nhất về vị trí địa lý của người dùng).
6. Ứng dụng React, khi cần dữ liệu động, gọi REST API tới domain Render qua HTTPS (sử dụng JWT nếu cần xác thực).
7. Backend trên Render xử lý request, truy vấn PostgreSQL, trả JSON theo chuẩn ApiResponse<T> về cho Frontend hiển thị.

6.6. Bảng tổng hợp cấu hình triển khai

Bảng 35: tổng hợp cấu hình triển khai

Tiêu chí	Backend (Render)	Frontend (Vercel)
Loại dịch vụ	Web Service — Docker runtime	Static / SPA (Vite framework preset)
Nhánh deploy	main	main (Production) / mọi nhánh khác (Preview)
Build command	docker build theo Dockerfile (2 stage)	npm install → npm run build
Output	Container image chạy dotnet api.dll	Thư mục dist/ (HTML/CSS/JS tĩnh)
Region	Singapore	Tự động chọn Edge gần người dùng nhất
Biến môi trường chính	ConnectionStrings__DefaultConnection, Jwt__Secret, Cors__AllowedOrigins...	VITE_API_URL, VITE_GOOGLE_CLIENT_ID
Health check	GET /health	Không cần (static hosting)
Domain mẫu	https://techshop-backend-8sfu.onrender.com	<a href="https://<project-name>.vercel.app">https://<project-name>.vercel.app
CI/CD	Auto-Deploy khi push lên main	Production Deploy (main) + Preview Deploy (mọi nhánh/PR)

6.7. Một số vấn đề thường gặp khi triển khai và cách xử lý

Bảng 36: Một số vấn đề thường gặp khi triển khai và cách xử lý

Vấn đề	Nguyên nhân thường gặp	Cách xử lý
Lỗi CORS khi Frontend gọi API	Domain Vercel chưa được thêm vào Cors__AllowedOrigins trên Render.	Cập nhật biến môi trường Cors__AllowedOrigins với đúng domain Production (và Preview nếu cần), sau đó deploy lại Backend.

Request đầu tiên rất chậm / timeout	Backend ở gói Free đã bị Render spin down do không có traffic trong 15 phút.	Chờ container khởi động lại (thường < 1 phút) hoặc thiết lập cron ping định kỳ tới /health.
Frontend build xong nhưng vẫn gọi localhost	Biến VITE_API_URL chưa được khai báo trên Vercel, hoặc đã đổi nhưng chưa Redeploy lại (Vite inject biến tại build-time).	Khai báo đúng VITE_API_URL trong Project Settings → Environment Variables, sau đó chạy lại Redeploy (không chỉ lưu biến).
Truy cập trực tiếp /laptop hoặc F5 bị lỗi 404	Vercel không biết cần fallback về index.html cho route do React Router xử lý phía client.	Thêm tệp vercel.json với rewrite "/(.*)" → "/index.html" như trình bày ở mục 6.3.4.
Lỗi 500 ngay sau khi deploy Backend	Thiếu hoặc sai biến môi trường bắt buộc (đặc biệt Jwt__Secret hoặc ConnectionStrings__DefaultConnection), hoặc CSDL production chưa chạy migration.	Kiểm tra tab Logs trên Render, rà soát lại đầy đủ Bảng 33, và chạy dotnet ef database update nhằm tới CSDL production.
Build Docker thất bại trên Render	Sai dockerContext hoặc dockerfilePath (Dockerfile cần COPY Backend/ từ gốc repo, không phải từ trong thư mục Backend/).	Đảm bảo dockerContext: . (thư mục gốc) và dockerfilePath: ./Dockerfile đúng như khai báo trong render.yaml.

6.8. Chạy toàn bộ hệ thống bằng Docker Compose (môi trường phát triển local)

Bên cạnh việc triển khai Backend lên Render và Frontend lên Vercel cho môi trường production, repository còn cung cấp tệp docker-compose.yml ở thư mục gốc nhằm dựng nhanh toàn bộ hệ thống ngay trên máy phát triển. Chỉ với một lệnh docker compose up --build, ba dịch vụ — cơ sở dữ liệu PostgreSQL, Backend .NET và Frontend React/Vite — sẽ được khởi chạy đồng thời và tự động kết nối với nhau, giúp lập trình viên kiểm thử toàn bộ luồng nghiệp vụ mà không cần cài đặt thủ công từng thành phần.

Bảng 37: Các dịch vụ định nghĩa trong docker-compose.yml

Dịch vụ	Image / Build	Cổng (host:container)	Vai trò
db	postgres:15-alpine	5432:5432	Cơ sở dữ liệu PostgreSQL cho môi trường phát triển local.
backend	Build từ ./Dockerfile	5000:10000	API .NET (ASP.NET Core); phụ thuộc dịch vụ db.
frontend	Build từ ./Frontend/Dockerfile	5173:5173	Giao diện React/Vite, hot-reload qua volume mã nguồn; phụ thuộc backend.

Dịch vụ db dùng image postgres:15-alpine làm cơ sở dữ liệu local — chỉ chứa dữ liệu phục vụ phát triển/kiểm thử và tách biệt hoàn toàn với cơ sở dữ liệu production đặt trên Supabase. Dịch vụ backend được build từ Dockerfile ở thư mục gốc, ánh xạ cổng 5000 (host) sang 10000 (container) và tự quyết định cấu hình dựa trên appsettings.json thay vì ghi đè biến môi trường trong compose. Dịch vụ frontend được build từ Frontend/Dockerfile, bật cờ CHOKIDAR_USEPOLLING và gắn volume mã nguồn để hỗ trợ hot-reload mượt mà khi chạy trong Docker. Việc khai báo depends_on giúp các dịch vụ khởi động theo đúng thứ tự: db → backend → frontend.

CHƯƠNG 7: KIỂM THỬ HỆ THỐNG

7.1. Công cụ và tài nguyên kiểm thử có sẵn

- Backend/TechShopApiTests.http — tập hợp các request mẫu (định dạng REST Client của VS Code/Visual Studio) cho toàn bộ endpoint, dùng để kiểm thử thủ công nhanh từng API riêng lẻ.
- Backend/TestFullApiFlow.ps1 — script PowerShell tự động chạy nối tiếp một luồng nghiệp vụ hoàn chỉnh (đăng nhập → lấy sản phẩm → thêm giỏ hàng → áp coupon → tạo đơn → lấy danh sách đơn...), hữu ích để kiểm tra nhanh sau mỗi lần thay đổi Backend hoặc sau khi deploy lên Render.
- Swagger UI (chỉ ở môi trường Development) — cho phép thử trực tiếp từng endpoint qua giao diện web, tự sinh từ thuộc tính route/DTO trong code.

7.2. Kịch bản kiểm thử luồng nghiệp vụ đầy đủ

Thứ tự kiểm thử được khuyến nghị (đúng theo trình tự phụ thuộc dữ liệu giữa các bước):

1. Chạy migration & khởi động Backend (hoặc xác nhận Backend trên Render đang ở trạng thái Live).
2. Đăng nhập với tài khoản khách hàng mẫu test@techshop.vn / Test@123.
3. Gọi API lấy danh sách sản phẩm (GET /api/products).
4. Gọi API lấy chi tiết một sản phẩm để lấy variantId cụ thể (GET /api/products/{slug}).
5. Thêm sản phẩm vào giỏ hàng (POST /api/cart/items).
6. Áp dụng mã giảm giá TECHSHOP10 (POST /api/cart/apply-coupon).
7. Tạo đơn hàng (POST /api/orders) và kiểm tra tồn kho đã bị trừ đúng.
8. Lấy lại danh sách đơn hàng của người dùng (GET /api/orders) để xác nhận đơn vừa tạo xuất hiện.
9. Thử hủy đơn (PATCH /api/orders/{id}/cancel) hoặc tạo thanh toán mock (POST /api/payments/vnpay/create), sau đó gọi callback để xác nhận trạng thái Payment/Order chuyển đúng sang Paid.
10. Đăng nhập với tài khoản quản trị admin@techshop.vn / Admin@123, kiểm thử các API quản trị: danh sách đơn hàng, điều chỉnh kho, xuất báo cáo Excel, quản lý người dùng.

7.3. Tài khoản kiểm thử mặc định

Bảng 38: Tài khoản kiểm thử mặc định

Vai trò	Email	Mật khẩu
Admin	admin@techshop.vn	Admin@123
Customer	test@techshop.vn	Test@123

7.4. Một số trường hợp kiểm thử tiêu biểu

Bảng 39: Một số trường hợp kiểm thử tiêu biểu

STT	Test case	Kết quả mong đợi
1	Đăng ký với email đã tồn tại	Trả lỗi success: false, error: "EMAIL_EXISTS", không tạo user mới.
2	Đăng nhập sai mật khẩu	Trả lỗi 401, không cấp accessToken/refreshToken.
3	Thêm vào giỏ khi chưa đăng nhập	Hệ thống tự tạo SessionId, trả về qua header X-Session-Id, giỏ hàng vẫn được lưu đúng theo session.
4	Áp coupon đã hết hạn hoặc vượt UsageLimit	Trả lỗi, không áp dụng giảm giá vào tổng tiền giỏ hàng.
5	Tạo đơn khi sản phẩm hết hàng (Quantity tồn kho < số lượng đặt)	Transaction rollback hoàn toàn, không tạo Order/OrderItem, không trừ kho.
6	Hủy đơn đã ở trạng thái Completed	Hệ thống chặn, trả lỗi, không cho đổi trạng thái.
7	Callback thanh toán với chữ ký sai	Backend từ chối callback (xác minh HMAC thất bại), không cập nhật trạng thái Payment.
8	Viết đánh giá cho sản phẩm chưa từng mua / đơn chưa Completed	Hệ thống chặn, không tạo Review.
9	Truy cập API quản trị (/api/admin/...) bằng tài khoản Customer	Trả lỗi 403 Forbidden do không đúng Role yêu cầu.
10	Gọi GET /health khi Backend đang chạy bình thường trên Render	Trả về 200 OK với nội dung { "status": "ok" }.

CHƯƠNG 8: KẾT QUẢ ĐẠT ĐƯỢC VÀ HẠN CHẾ

8.1. Kết quả đạt được

- Hoàn thiện toàn bộ 67 endpoint REST API cho các nghiệp vụ: xác thực, danh mục/sản phẩm, giỏ hàng, đơn hàng, thanh toán, khuyến mãi, đánh giá và toàn bộ phân hệ quản trị.
- Hoàn thiện giao diện Frontend cho cả luồng khách hàng (mua sắm, giỏ hàng, thanh toán, tài khoản) và luồng quản trị (7 trang quản trị với layout riêng).
- Thiết kế cơ sở dữ liệu chuẩn hóa với 21 thực thể, đầy đủ ràng buộc khóa ngoại, chỉ mục duy nhất và quy tắc xóa dữ liệu rõ ràng.
- Áp dụng thành công mô hình bảo mật JWT hai lớp (access + refresh token), mã hóa mật khẩu BCrypt và xác minh chữ ký HMAC cho callback thanh toán.
- Container hóa Backend bằng Docker và triển khai thành công, vận hành thực tế trên Render với endpoint công khai và cơ chế Health Check tự động.
- Xây dựng quy trình triển khai Frontend chi tiết lên Vercel, bao gồm cấu hình biến môi trường, SPA fallback và đồng bộ CORS hai chiều với Backend.
- Thiết lập được cơ chế CI/CD đơn giản nhưng hiệu quả: mọi thay đổi push lên main đều được tự động build và deploy lại trên cả hai nền tảng.

8.2. Hạn chế hiện tại

Bảng 40: Hạn chế hiện tại

Hạng mục	Tình trạng hiện tại
Cổng thanh toán VNPAY/MoMo/VietQR/ZaloPay	Đang ở dạng mock — tự sinh URL & callback có ký HMAC, chưa kết nối API thật của các cổng thanh toán.
Dịch vụ Email	EmailService (email xác nhận đơn hàng/thanh toán) vẫn chỉ ghi log ra console; chức năng "Quên mật khẩu" đã gửi email thật qua Brevo API.
Duyệt đánh giá	Đánh giá mới luôn ở trạng thái ẩn (isVisible = false); chưa có API/giao diện quản trị riêng để Admin duyệt hiển thị, hiện phải chỉnh trực tiếp trong CSDL.
Hộp nhất giỏ hàng (Cart merge)	Khi khách đang có giỏ hàng theo sessionId rồi đăng nhập, hệ thống chưa tự động gộp giỏ hàng khách vào giỏ hàng của tài khoản.
Cập nhật sản phẩm dạng nested	API cập nhật sản phẩm hiện chỉ cập nhật thông tin chính, chưa hỗ trợ cập nhật đồng thời danh sách ảnh/biến thể/thông số lồng nhau trong một request.
Validate file upload	Việc tải ảnh sản phẩm chưa kiểm tra chặt phần mở rộng/Content-Type, có thể là điểm cần tăng cường bảo mật.
Kiểm thử tự động	Chưa có bộ unit test/integration test tự động (mới có kiểm thử thủ công qua .http và script PowerShell).
vercel.json	Repository đã bổ sung tệp Frontend/vercel.json với quy tắc rewrite về index.html, xử lý lỗi 404 ở các route con của React Router (xem mục 6.3.4).

CHƯƠNG 9: HƯỚNG PHÁT TRIỂN

Dựa trên các hạn chế đã nêu ở Chương 8, đề tài có thể được phát triển tiếp theo các hướng sau:

1. Tích hợp API thật của VNPay và MoMo, thay thế hoàn toàn lớp `PaymentGatewayService` mock hiện tại.
2. Tích hợp dịch vụ gửi email thật (SMTP, SendGrid, hoặc Amazon SES) để gửi email xác nhận đơn hàng/thanh toán/đổi trạng thái cho khách hàng.
3. Xây dựng API và giao diện quản trị cho phép Admin duyệt/ẩn đánh giá sản phẩm trực tiếp, không cần can thiệp CSDL thủ công.
4. Bổ sung cơ chế tự động hợp nhất giỏ hàng khách vào giỏ hàng tài khoản ngay sau khi đăng nhập thành công.
5. Viết bộ unit test (xUnit/NUnit) cho các Service/Controller quan trọng và tích hợp vào pipeline CI để tự động kiểm tra trước khi deploy.
6. Nâng cấp gói Render/Vercel lên bản trả phí khi đưa vào sử dụng thực tế, để loại bỏ hiện tượng cold start và có thêm tài nguyên xử lý đồng thời.
7. Thiết lập custom domain riêng cho cả Frontend (Vercel) và Backend (Render) để hệ thống có địa chỉ chuyên nghiệp, dễ ghi nhớ hơn.

CHƯƠNG 10: KẾT LUẬN

Qua quá trình thực hiện đề tài, nhóm đã xây dựng hoàn chỉnh một hệ thống thương mại điện tử TechShop với đầy đủ các nghiệp vụ cốt lõi của một website bán hàng thực tế: từ quản lý danh mục/sản phẩm, giỏ hàng, đặt hàng, thanh toán, khuyến mãi, đánh giá sản phẩm, cho tới một phân hệ quản trị tương đối đầy đủ phục vụ vận hành (quản lý kho, đơn hàng, người dùng và báo cáo thống kê).

Về mặt công nghệ, đề tài cho phép nhóm thực hành toàn diện một quy trình phát triển phần mềm hiện đại: thiết kế cơ sở dữ liệu quan hệ chuẩn hóa, xây dựng RESTful API với ASP.NET Core và Entity Framework Core, phát triển giao diện Single Page Application với React và hệ sinh thái thư viện hiện đại (Zustand, React Query, React Hook Form), áp dụng các cơ chế bảo mật then chốt (JWT, BCrypt, HMAC, CORS), và đặc biệt là thực hành container hóa ứng dụng bằng Docker cùng triển khai thực tế lên hai nền tảng cloud miễn phí phổ biến hiện nay là Render và Vercel theo mô hình tách triển khai (split deployment).

Việc Backend đã được triển khai thành công và đang hoạt động công khai trên Render là minh chứng cụ thể cho tính khả thi của kiến trúc và quy trình DevOps mà nhóm đã thiết lập; quy trình triển khai Frontend lên Vercel được trình bày chi tiết trong báo cáo (Chương 6) cũng đã sẵn sàng để áp dụng ngay, hoàn tất việc đưa toàn bộ hệ thống TechShop vào vận hành công khai trên Internet.

Những hạn chế còn tồn đọng (cổng thanh toán mock, dịch vụ email mock, chưa có kiểm thử tự động...) đã được nhận diện rõ ràng và là cơ sở cho các hướng phát triển tiếp theo, giúp hệ thống ngày càng hoàn thiện và tiến gần hơn tới một sản phẩm có thể đưa vào sử dụng thực tế.

TÀI LIỆU THAM KHẢO

1. Microsoft Learn — ASP.NET Core documentation: <https://learn.microsoft.com/aspnet/core>
2. Microsoft Learn — Entity Framework Core documentation: <https://learn.microsoft.com/ef/core>
3. Npgsql — .NET Access to PostgreSQL: <https://www.npgsql.org>
4. React — Official documentation: <https://react.dev>
5. Vite — Official documentation: <https://vite.dev>
6. React Router — Official documentation: <https://reactrouter.com>
7. TanStack Query — Official documentation: <https://tanstack.com/query>
8. Render — Official documentation: <https://render.com/docs>
9. Vercel — Official documentation: <https://vercel.com/docs>
10. Docker — Official documentation: <https://docs.docker.com>
11. JWT.io — Introduction to JSON Web Tokens: <https://jwt.io/introduction>
12. Repository đồ án: <https://github.com/PiupiuTenshi/TechWeb-2026>